Investigating Double Descent: Multiple Regression Analysis on the Friedman Datasets

Angie McGraw (MSCS Year 1) Anjali Saravanan (MSDS Year 1)
Caolan Disini (MSCS Year 2) Nakul Rao (MSCS Year 2)

20 April 2026

1 Introduction

The classical bias-variance tradeoff predicts that as model complexity increases, a model will reach an interpolation threshold where it memorizes training data and fails to generalize. Double descent challenges this: past the threshold, further increases in complexity can cause test error to decrease again, suggesting overparameterized models can generalize. While double descent has been extensively studied in deep learning classification, little is known about its behavior in simpler regression models.

We investigate double descent across four models of increasing complexity: (1) polynomial regression, (2) random feature regression, (3) kernel ridge regression, and (4) a single-layer neural network, evaluated on three synthetic Friedman regression datasets with known generating functions and tunable noise.

We find that double descent is dataset- and model-dependent: random feature regression and the neural network exhibit clear double descent on Friedman 1, while polynomial regression and kernel ridge regression follow the classical tradeoff. Friedman 3 was the most challenging across all models, suggesting that data complexity and model class jointly determine whether double descent appears in regression settings.

2 Background

Prior work on double descent has focused on overparameterized deep learning classification models [4], leaving a gap in regression settings. Allerbo et al. [2] theoretically explain why constant-bandwidth Kernel Ridge Regression (KRR) cannot exhibit double descent, though their proposed fix requires iterative kernel recomputation, making it impractical for standard pipelines. Random Feature Regression offers an efficient way to control complexity and observe double descent, but depends on randomly constructed features [6]. Schaeffer et al. [5] identify three conditions for double descent in linear regression: (1) insufficient training data to explain certain feature relationships, (2) test data lying within those underrepresented relationships, and (3) irreducible noise. However, their analysis is limited to linear models.

3 Summary of the Data

We use the synthetic Friedman datasets (Friedman-1, 2, and 3), originally designed to evaluate non-linear regression performance [1]. Each dataset is generated from a complex non-linear equation

and includes both relevant and irrelevant features to test how well models separate signal from noise.

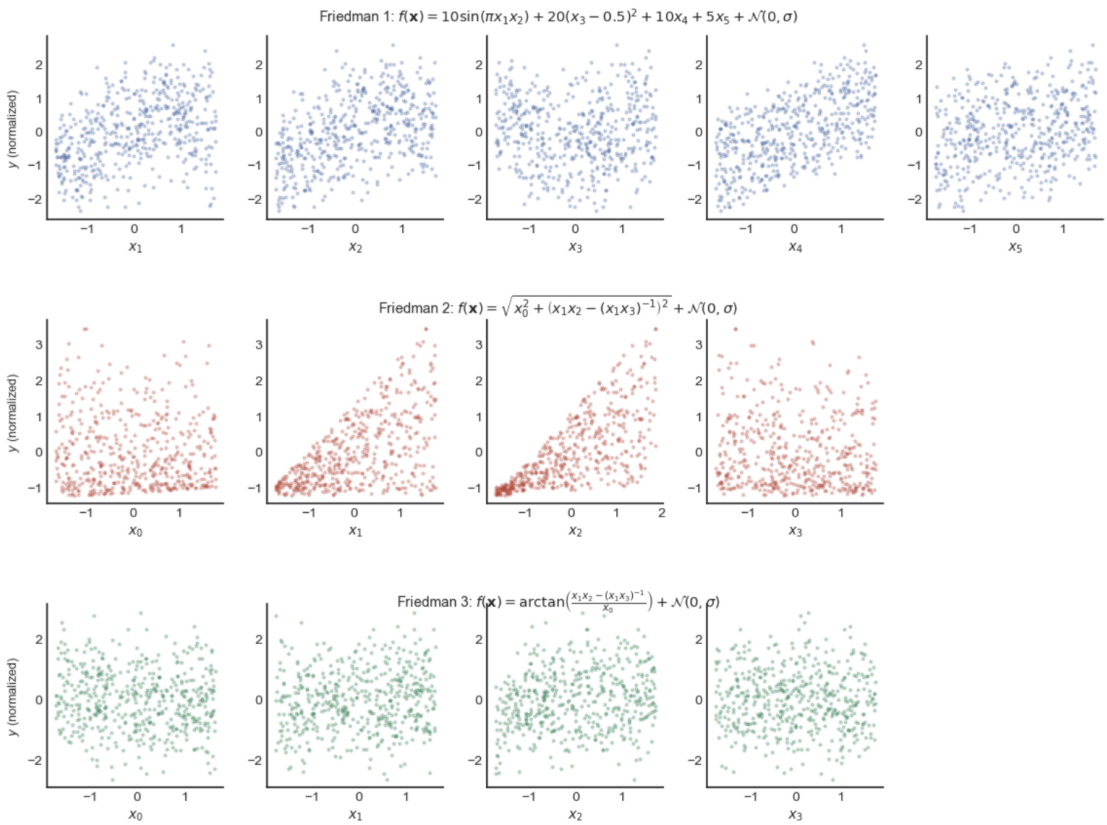

Figure 1: Scatter plots of normalized target y against each informative input feature for the three Friedman datasets ($n_{\text{train}} = 500$, NOISE $= 1.0$). For Friedman 1, the first 5 features are relevant; the remainder are distractors.

4 Methods

We use three regression models and a neural network baseline to investigate double descent.

4.1 Polynomial Regression

Polynomial regression expands input x into higher-degree terms and fits a linear model in the transformed space:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d \tag{1}$$

Coefficients are estimated by ordinary least squares:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \tag{2}$$

The degree d controls model complexity; higher degrees increase flexibility but risk overfitting and numerical instability.

4.2 Random Feature Regression (RFR)

RFR approximates kernel methods by mapping inputs into a lower-dimensional random feature space. We use Random Fourier Feature Regression (RFFR), which approximates the Gaussian kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right) \tag{3}$$

Via the Bochner Theorem [6], this shift-invariant kernel is generalized through its Fourier transform:

$$p(\boldsymbol{\omega}) = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} e^{-j\langle \boldsymbol{\omega}, \boldsymbol{\tau} \rangle}\, k(\boldsymbol{\tau})\, d\boldsymbol{\tau} \tag{4}$$

where i.i.d. Monte Carlo samples $\boldsymbol{\omega}_1, \ldots, \boldsymbol{\omega}_D \sim p$ are used to construct the feature map.

4.3 Kernel Ridge Regression (KRR)

KRR replaces dot products with a kernel function, yielding a model that is nonlinear in data but linear in parameters. Given n training pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$:

$$f(\mathbf{x}) = \sum_{i=1}^{n} c_i\, k(\mathbf{x}, \mathbf{x}_i) \tag{5}$$

Allerbo (2025) [2] showed that KRR with decreasing bandwidth (KGD-D) can exhibit double descent. We test whether standard, fixed-bandwidth KRR does so. The closed-form dual solution is:

$$\mathbf{c} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y} \tag{6}$$

with predictions $f(\mathbf{x}^*) = \mathbf{k}(\mathbf{x}^*, \mathbf{X})^\top (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$. Here λ controls the bias-variance tradeoff: large λ underfits; small λ approaches interpolation $\mathbf{f} = \mathbf{K}^{-1}\mathbf{y}$.

Kernel Choice. We used the Gaussian (RBF) kernel $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ with bandwidth set via the median heuristic: $\gamma = 1/(2 \cdot \text{median}\{\|\mathbf{x}_i - \mathbf{x}_j\|^2 : i < j\})$. Unlike KGD-D, γ is fixed once.

Complexity Sweep. We swept $\lambda(c) = 10^{-c/10}$ for $c \in \{1, \ldots, 79\}$, producing values from $\sim 7.94 \times 10^{-1}$ to $\sim 1.26 \times 10^{-8}$, divided into four regimes: heavy ($\lambda > 10^{-1}$), moderate ($10^{-2} < \lambda \leq 10^{-1}$), light ($10^{-4} < \lambda \leq 10^{-2}$), and near-interpolation ($\lambda \leq 10^{-4}$).

Label Corruption. Following Nakkiran et al. [4], we applied 15% label corruption: for each run, a random subset $S \subset \{1, \ldots, n\}$ with $|S| = \lfloor 0.15n \rfloor$ had labels replaced by values sampled uniformly from $\{y_1, \ldots, y_n\}$. Each λ was evaluated over 20 independent runs.

4.4 Neural Network

Our neural network is a single hidden layer MLP with ReLU activation:

$$\hat{y} = \mathbf{W}_2\, \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + b_2 \tag{7}$$

trained to minimize MSE via full-batch Adam ($\eta = 10^{-3}$, 10,000 epochs):

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{8}$$

Complexity Sweep. The number of hidden units h varies from 1 to 400, giving parameter count $p = h(d+2)+1$. The interpolation threshold is $h^* = \lfloor n/(d+2) \rfloor$: for Friedman 1 ($d = 10$), $h^* = 41$ ($p \approx 493$); for Friedman 2 and 3 ($d = 4$), $h^* = 83$ ($p \approx 499$). The grid has fine resolution near h^*, and each value is evaluated over 20 runs ($N_{\text{RUNS}} = 20$).

Label Corruption. An ablation over optimizer (Adam vs. SGD) and corruption rate (0% vs. 15%) showed that label corruption is unnecessary for double descent when dataset noise (NOISE = 1.0) is present. We therefore use Adam with no corruption (Appendix A).

5 Implementation

All source code is available at https://github.com/angiemcgraw/double-descent-friedman/tree/main. The project is implemented in Python and organized as follows.

Shared Infrastructure. Datasets are generated with scikit-learn's `make_friedman{1,2,3}` ($n_{\text{train}} = 500$, $n_{\text{test}} = 1000$, NOISE = 1.0) and normalized to zero mean and unit variance using training set statistics. Distinct random seeds with offsets are used per dataset to prevent train/test overlap. The main training loop in `experiments/run.py` takes command-line arguments `--model`, `--dataset`, `--optimizer`, and `--corruption`. For each complexity value, $N_{\text{RUNS}} = 20$ independent runs execute in parallel via `joblib` with `n_jobs=2`. Seeds are constructed as $42 + \text{run} + 1000 \cdot h$ to ensure independence across both runs and complexity values. Results are saved as summary CSV files, per-run and averaged.

Polynomial Regression. Following Schaeffer et al. [5], who identified three conditions for double descent, this experiment replicates those conditions on the Friedman datasets. The model uses NumPy and scikit-learn: inputs are expanded to degree-3 (Friedman 1) or degree-7 (Friedman 2, 3) polynomials, then standardized via `StandardScaler` for numerical stability across high-degree terms. The first p features are selected to control complexity (interpolation threshold $p = 100$). Without regularization, OLS is applied via `np.linalg.lstsq`; with ridge, coefficients are computed via `np.linalg.solve` with penalty α. Active coefficient count p is tracked at each complexity level.

Random Feature Regression. RFFR uses the RFFLearn library, a scikit-learn wrapper for random Fourier features. The random matrix kernel dimension matches the feature size of 10 for Friedman-1, with a fixed bandwidth of $\sigma = 0.5$ to accelerate the onset of overfitting and double descent. Model attributes for Friedman-2 and 3 are kept identical to maintain parity across datasets.

Kernel Ridge Regression. KRR uses scikit-learn's `KernelRidge` class, which solves $(\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{y}$ in closed form with the median-heuristic RBF bandwidth. Because the solution is analytic, there is no training loop—λ is the sole hyperparameter. As $\lambda \to 0$, $(\mathbf{K} + \lambda\mathbf{I})$ becomes nearly singular; non-finite predictions (particularly on Friedman 2) are clamped to large finite values so downstream MSE remains well-defined. The complexity-to-alpha mapping is handled in `get_model()`, and a KRR-specific alpha-axis plot with inverted log scale is generated alongside the default complexity-axis plot.

Neural Network. The neural network is implemented in PyTorch (`models/neural_network.py`) using the shared `BaseModel` class. It is a single hidden layer MLP built with `nn.Sequential`: input dimension d, hidden dimension h with ReLU, and a scalar output. Weights use He initialization

via `kaiming_uniform_`, sampling from $\mathcal{N}(0, 2/d)$; biases are set to zero. Full-batch gradient descent trains all 500 samples per epoch. The `--optimizer` flag selects Adam or SGD; Adam ($\eta = 10^{-3}$, 10,000 epochs) produced the best double descent curves.

6 Results

6.1 Polynomial Regression

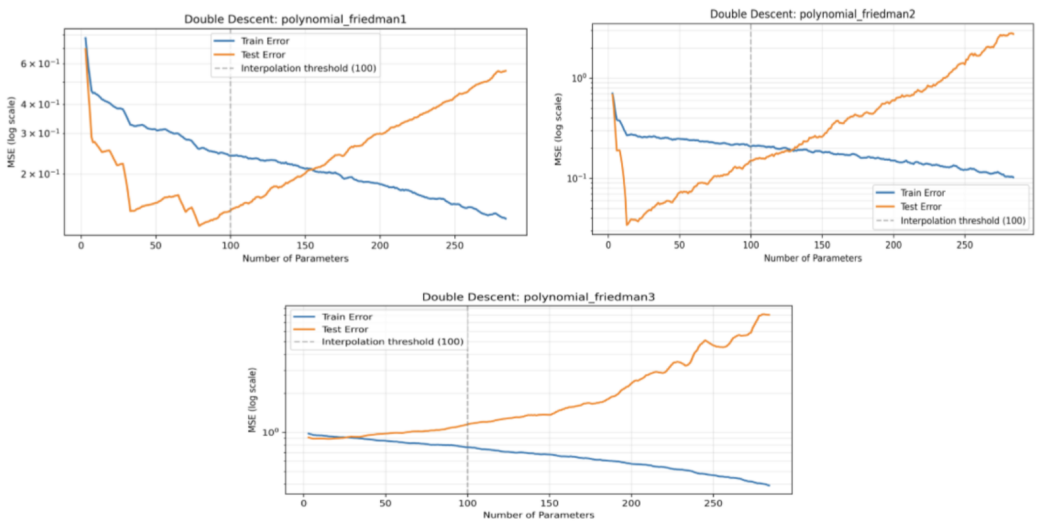

Figure 2: Polynomial regression results. Friedman 1: min train/test error = 0.068/0.016; Friedman 2: 0.239/0.029; Friedman 3: 0.883/0.890.

Double descent was not observed on any dataset. Polynomial regression lacks the capacity to interpolate training data smoothly in the overparameterized regime. This confirms that double descent is not universal—even when theoretical conditions are met, the model structure must also support it.

6.2 Random Feature Regression

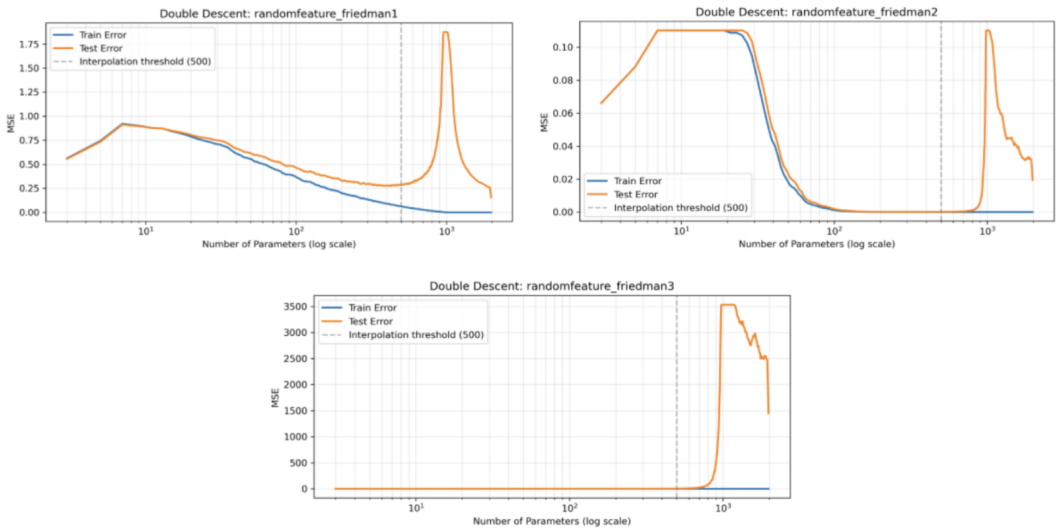

Figure 3: Random feature regression results. Friedman 1: min train/test error = 0.000/0.256; Friedman 2: 0.000/0.000; Friedman 3: 0.000/0.893.

Clear double descent appears on Friedman-1, where parameters heavily outnumber samples. Friedman-2 trends toward a second descent that may resolve with further overparameterization. RFFR's explicit random feature sampling makes it well-suited for observing the phenomenon.

6.3 Kernel Ridge Regression

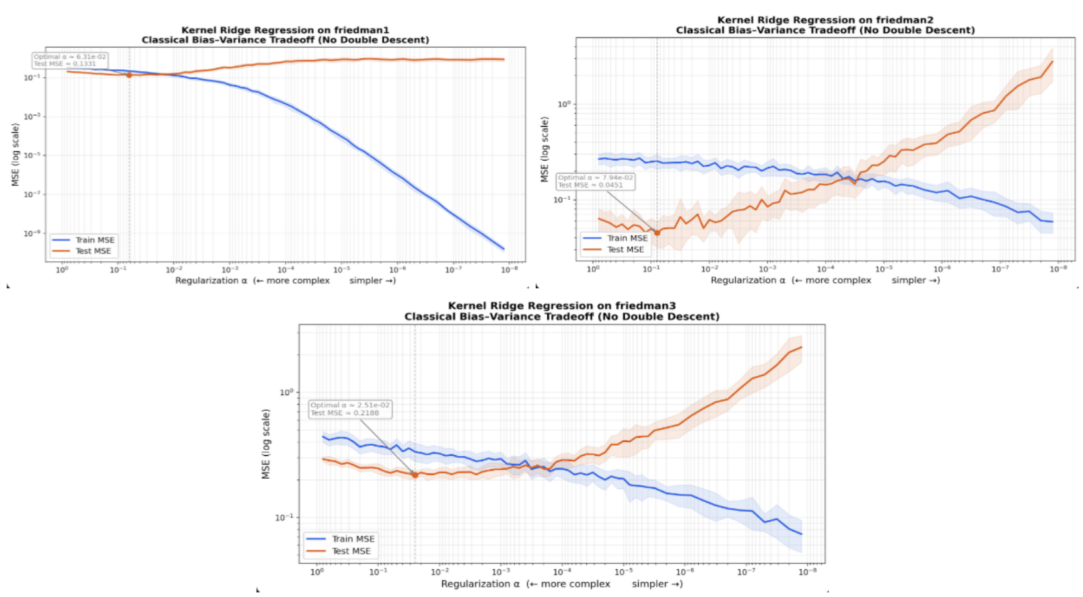

Figure 4: Kernel ridge regression results. Friedman 1: min train/test error = 0.000/0.133; Friedman 2: 0.059/0.045; Friedman 3: 0.074/0.219.

Across all three datasets, KRR exhibited only the classical bias-variance tradeoff. No double descent was observed. Test error decreased as λ shrank from the heavy-regularization regime, reached an optimum in the moderate regime, and then rose monotonically through light regularization and near-interpolation.

The Role of the Regularization Parameter. The four λ regimes reveal a clear progression in generalization behavior. In the *heavy-regularization* regime ($\lambda > 10^{-1}$), the model underfits: train MSE is high across all datasets (0.28 on Friedman 1, 0.26 on Friedman 2, 0.41 on Friedman 3), and notably exceeds test MSE. This inverted gap is a consequence of 15% label corruption inflating training targets relative to clean test labels; the train–test gap is negative (e.g., -0.11 on Friedman 1), indicating the model is too constrained to memorize corrupted labels.

In the *moderate-regularization* regime ($10^{-2} < \lambda \leq 10^{-1}$), all three datasets achieve their optimal test MSE: 0.133 for Friedman 1 at $\lambda = 6.31 \times 10^{-2}$, 0.045 for Friedman 2 at $\lambda = 7.94 \times 10^{-2}$, and 0.219 for Friedman 3 at $\lambda = 2.51 \times 10^{-2}$. The train–test gap shrinks toward zero, indicating a balanced fit.

The *light-regularization* regime ($10^{-4} < \lambda \leq 10^{-2}$) marks the onset of overfitting. On Friedman 1, test MSE jumps to 0.34 while train MSE drops to 0.054; the train–test gap flips positive and widens. The standard deviation of test MSE also increases, reflecting growing instability across runs.

In the *near-interpolation* regime ($\lambda \leq 10^{-4}$), overfitting becomes severe. Train MSE drops to near zero across all datasets while test MSE reaches its highest values (0.82 on Friedman 1, 0.61 on Friedman 2, 0.71 on Friedman 3). This instability is a direct consequence of the kernel matrix becoming ill-conditioned: small differences in corrupted labels produce widely different interpolating solutions. The effective degrees of freedom approach 500, confirming full interpolation.

Critically, test error never recovers after this threshold. Unlike neural networks and RFR, which can grow model capacity, $\mathbf{K}$ is fixed at 500×500 regardless of λ. As $\lambda \to 0$, the solution converges to a single interpolator $\mathbf{K}^{-1}\mathbf{y}$ with no additional degrees of freedom to find a smoother fit. There is no overparameterized regime—only an increasingly ill-conditioned interpolation.

6.4 Neural Network

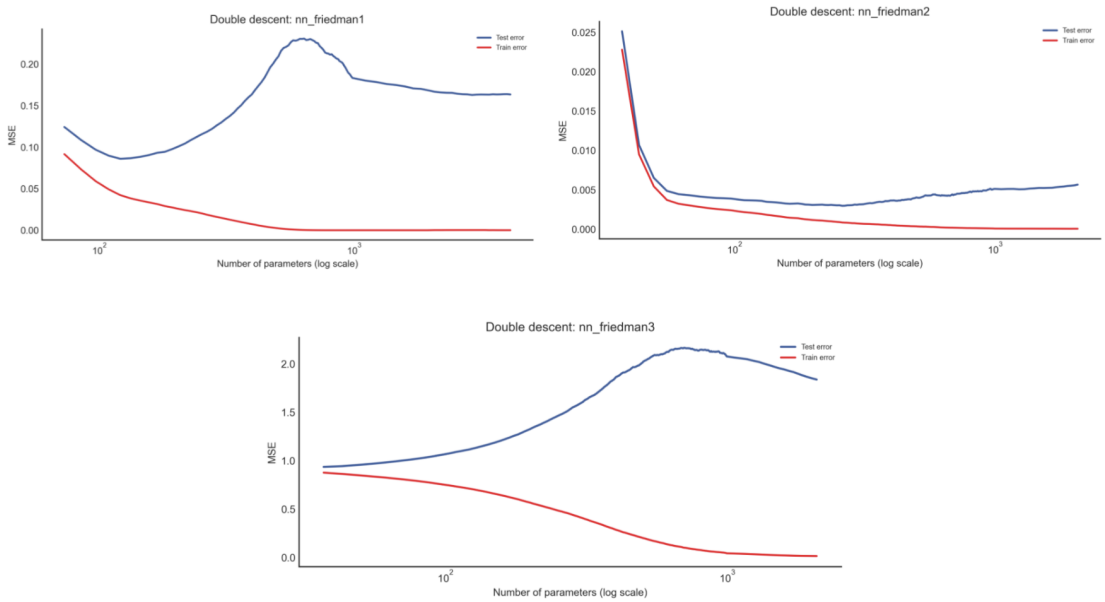

Figure 5: Neural network results. Friedman 1: min train/test error = 0.000/0.077; Friedman 2: 0.000/0.003; Friedman 3: 0.014/0.899.

The neural network produced the cleanest double descent on Friedman 1 (test error 0.077). Friedman 2 showed no double descent; the model appears to memorize its multiplicative structure, with nearly identical train and test curves. Friedman 3 test error rose then fell with no clear peak (train error 0.014, test error 0.899). These results reinforce that double descent requires a dataset complex enough for small models to struggle and large enough models to interpolate.

7 Discussion

Double descent is not a universal property of overparameterized models—it is both model and dataset dependent. Of the four models tested, Random Feature Regression and the Neural Network exhibited the characteristic test error peak and recovery near the interpolation threshold, consistently aligned where $p \approx n$ [4]. Polynomial Regression and KRR followed the classical trade-off, showing that double descent requires a model family whose optimizer can select generalizing solutions beyond the interpolation threshold. Dataset structure also matters: Friedman 1 produced the cleanest curves, while Friedman 3's arctan nonlinearity created sharp transitions hostile to all model families.

Several extensions are natural. Friedman 2 and 3 warrant exploration at greater complexity ranges to confirm whether the second descent fully resolves. Implementing KGD-D [2] would test whether double descent can be induced in kernel regression. Sweeping label corruption from 0% to 30% would reveal how noise intensity interacts with the interpolation threshold. Finally, ensemble methods (random forests, gradient boosting) would clarify which model properties are necessary for the phenomenon to emerge.

References

[1] J. H. Friedman, "Multivariate adaptive regression splines," *The Annals of Statistics*, vol. 19, no. 1, pp. 1–67, 1991.

[2] O. Allerbo, "Changing the kernel during training leads to double descent in kernel regression," *arXiv preprint arXiv:2311.01762*, 2025.

[3] M. Lafon and A. Thomas, "Understanding the double descent phenomenon in deep learning," *arXiv preprint arXiv:2403.10459*, 2024.

[4] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever, "Deep double descent: Where bigger models and more data hurt," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2021, no. 12, p. 124003, 2021.

[5] R. Schaeffer, M. Khona, Z. Robertson, A. Boopathy, K. Pistunova, J. W. Rocks, I. R. Fiete, and O. Koyejo, "Double descent demystified: Identifying, interpreting & ablating the sources of a deep learning puzzle," *arXiv preprint arXiv:2303.14151*, 2023.

[6] W. Rudin, *Fourier Analysis on Groups*. John Wiley & Sons, 1991.

A Neural Network Ablation Experiment

To determine the necessary conditions for double descent in the neural network, an ablation was conducted on Friedman 1, varying optimizer (Adam vs. SGD) and label corruption (0% vs. 15%) with $N_{\text{RUNS}} = 3$, epochs = 3,000, and NOISE = 0.1. Results are in `figures/nn/initial_small_trial_runs_noise_0.1`. Adam with and without corruption showed double descent; we adopted Adam with no corruption for all final runs.

Statement of Contributions

Anjali Saravanan implemented polynomial regression. Caolan Disini implemented random feature regression. Nakul Rao implemented kernel ridge regression. Angie McGraw implemented the neural network. All members contributed to the shared codebase, presentations, and final report.